



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**
título del TFG



Presentado por Nombre del alumno
en Universidad de Burgos — 20 de junio
de 2026

Tutor: nombre tutor



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. nombre tutor, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Nombre del alumno, con DNI dni, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 20 de junio de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. nombre tutor

D. nombre co-tutor

Resumen

El presente Trabajo de Fin de Grado describe el diseño y desarrollo de AgroSkopos, una aplicación web integral destinada a la monitorización, telemetría y control remoto de un vehículo agrícola autónomo. Ante la necesidad actual de modernizar las explotaciones agrarias mediante herramientas de agricultura de precisión, este proyecto propone una solución centralizada basada en una arquitectura cliente-servidor de baja latencia.

El sistema centraliza las operaciones críticas del vehículo ofreciendo un mapa interactivo para el seguimiento de rutas, herramientas para la gestión del historial de misiones y un panel de control manual con soporte para transmisión de vídeo en vivo. Para validar la fiabilidad de la plataforma como paso previo a su integración con *hardware* físico, se ha implementado un motor de simulación nativo en el servidor. Este simulador emula el comportamiento cinemático y energético del robot, garantizando la correcta recepción y procesamiento en tiempo real de variables críticas como las coordenadas GPS, la velocidad y el nivel de batería.

A nivel técnico, la plataforma ha sido construida utilizando tecnologías modernas y modulares, destacando el uso de React y Zustand en el *frontend* para una gestión eficiente del estado, y Node.js junto con PostgreSQL en el *backend* para la lógica de negocio y persistencia de datos. Todo el ecosistema ha sido contenerizado mediante Docker, asegurando un despliegue robusto, escalable y fácilmente adaptable a futuros entornos de producción.

Descriptores

Telemetría, Agricultura de precisión, Robótica agrícola, Aplicación web, Tiempo real, Simulación, React, Node.js.

Abstract

This Final Degree Project describes the design and development of AgroSkopos, a comprehensive web application aimed at the monitoring, telemetry, and remote control of an autonomous agricultural vehicle. Given the current need to modernize farming operations through precision agriculture tools, this project proposes a centralized solution based on a low-latency client-server architecture.

The system centralizes the vehicle's critical operations by offering an interactive map for route tracking, tools for mission history management, and a manual control panel with support for live video streaming. To validate the platform's reliability as a preliminary step prior to its integration with physical *hardware*, a native simulation engine has been implemented on the server. This simulator emulates the kinematic and energetic behavior of the robot, ensuring the correct real-time reception and processing of critical variables such as GPS coordinates, speed, and battery level.

On a technical level, the platform has been built using modern and modular technologies, highlighting the use of React and Zustand on the *frontend* for efficient state management, and Node.js alongside PostgreSQL on the *backend* for business logic and data persistence. The entire ecosystem has been containerized using Docker, ensuring a robust, scalable deployment that is easily adaptable to future production environments.

Keywords

Telemetry, Precision agriculture, Agricultural robotics, Web application, Real-time, Simulation, React, Node.js.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
Introducción	1
Objetivos del proyecto	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
Conceptos teóricos	7
3.1. Secciones	7
3.2. Referencias	7
3.3. Imágenes	8
3.4. Listas de ítems	8
3.5. Tablas	9
Técnicas y herramientas	11
4.1. Metodologías y técnicas de gestión	11
4.2. Patrón de arquitectura	12
4.3. Tecnologías y <i>frameworks</i> principales	14
4.4. Gestión de código y entorno de desarrollo	19
Aspectos relevantes del desarrollo del proyecto	21
5.1. Motivación del proyecto	21
5.2. Inicio del proyecto y evolución del alcance	22

5.3. Planificación	22
5.4. Adquisición de conceptos técnicos	23
5.5. Desarrollo del proyecto	25
5.6. Testing y Calidad de Código	26
5.7. Despliegue	27
Trabajos relacionados	29
Conclusiones y líneas de trabajo futuras	31
7.1. Conclusiones	31
7.2. Líneas de trabajo futuras	32
Bibliografía	37

Índice de figuras

3.1. Autómata para una expresión vacía	8
--	---

Índice de tablas

3.1. Herramientas y tecnologías utilizadas en cada parte del proyecto	9
---	---

Introducción

La aplicación cubre los procesos esenciales de operación: desde el control de misiones y la visualización telemétrica, hasta la supervisión en vivo mediante el flujo de cámara y el control manual del vehículo. Al integrar estas funcionalidades en una plataforma modular, se pretende no solo agilizar los flujos de trabajo, sino ofrecer una herramienta que se adapte con facilidad a las exigencias de la agricultura de precisión.

Un pilar diferenciador de este proyecto es su arquitectura orientada a la baja latencia y el procesamiento dinámico de datos. Aunque el sistema fue concebido originalmente para su integración con una plataforma robótica física desarrollada en colaboración con el área de ingeniería electrónica, la solución actual emplea un robusto simulador de telemetría nativo. Este componente permite validar la lógica de control y monitorizar variables críticas —como el nivel de batería, las coordenadas GPS y el estado de carga— garantizando la seguridad operativa y la fiabilidad del *software* como paso previo a una futura implementación en *hardware* real.

Este documento constituye la memoria principal del proyecto, donde se detallan los objetivos, conceptos teóricos, técnicas empleadas y los aspectos más relevantes del desarrollo. La documentación técnica completa se complementa con el historial de desarrollo disponible públicamente en el repositorio de *GitHub* del proyecto:

<https://github.com/andresveracachoo/aplicacion-robot-agricola>

El progreso y la organización de las tareas y *scripts* del proyecto han sido gestionados mediante la herramienta de gestión ágil Zube, permitiendo una trazabilidad clara del flujo de trabajo. Asimismo, para garantizar la

robustez y mantenibilidad del *software*, se ha integrado un análisis estático de código realizado por SonarCloud [24], cuyos reportes de calidad técnica pueden visualizarse en:

https://sonarcloud.io/project/overview?id=AndresVeraCachoo_aplicacion-robot-agricola

Objetivos del proyecto

Este capítulo tiene como propósito definir el alcance y las metas fundamentales que guían el desarrollo de AgroSkopos. El proyecto surge de la necesidad de proporcionar una interfaz de usuario avanzada para la gestión de un vehículo autónomo en entornos agrícolas, priorizando la precisión en la telemetría y la fiabilidad en el control remoto.

2.1. Objetivo General

El objetivo principal de este Trabajo de Fin de Grado es el diseño, implementación y validación de una plataforma web centralizada para la monitorización, telemetría y control de un robot agrícola autónomo. La solución debe actuar como un visor de precisión, permitiendo al operador supervisar el estado del vehículo en tiempo real y gestionar misiones de trabajo de manera eficiente, garantizando la integridad operativa mediante un entorno de simulación.

2.2. Objetivos Específicos

Para alcanzar el objetivo general propuesto, se han definido una serie de metas específicas divididas en tres vertientes: funcionales, técnicas y de calidad y gestión.

Objetivos Funcionales

- **Visualización Geográfica:** Integrar un sistema de mapas interactivos basado en Leaflet [1] que permita el seguimiento preciso de la ubicación del robot y la representación de rutas de misión.
- **Panel de Telemetría:** Desarrollar un panel dinámico que procese y muestre variables críticas de estado, tales como el nivel de batería, la velocidad de avance y las coordenadas GPS.
- **Control Remoto y Vídeo:** Implementar una interfaz de control manual para la operación directa del vehículo, integrando un flujo de vídeo en baja latencia para la supervisión visual.
- **Gestión de Misiones:** Crear un sistema de persistencia para el historial de misiones, permitiendo la consulta de datos históricos y el análisis de rendimiento de jornadas anteriores.

Objetivos Técnicos

- **Arquitectura de Baja Latencia:** Configurar un sistema de comunicación cliente-servidor mediante técnicas de consultas periódicas u optimizadas (*polling* a intervalos de 2 segundos) para asegurar que la telemetría refleje el estado real del vehículo.
- **Gestión de Estado Global:** Utilizar Zustand [20] para la administración del estado de la aplicación en el *frontend*, garantizando una reactividad fluida sin las sobrecargas de rendimiento asociadas a otras librerías.
- **Simulación Cinemática:** Desarrollar un motor de simulación nativo en el *backend* que emule el comportamiento físico y energético del robot, permitiendo validar la lógica de la aplicación sin dependencia inicial del *hardware* físico.
- **Contenerización:** Orquestar todo el ecosistema de *software* (interfaz, lógica de servidor y base de datos) mediante Docker [7] para asegurar la portabilidad, escalabilidad y un despliegue robusto del sistema.

Objetivos de Gestión y Calidad

- **Metodología Ágil (Scrum [22]):** Implementar un marco de trabajo basado en Scrum [22], estructurado en iteraciones o *sprints* de dos

semanas. Este enfoque permite gestionar la variabilidad en mi disponibilidad y garantiza una validación continua de los requisitos mediante reuniones de revisión periódicas con el tutor.

- **Pruebas Automatizadas (*Testing*):** Diseñar e implementar una estrategia integral de pruebas automáticas, abarcando pruebas unitarias para aislar la lógica de componentes y controladores, así como pruebas de integración (*End-to-End*) sobre entornos reales. El objetivo es garantizar la estabilidad del sistema, prevenir regresiones y habilitar su ejecución automática mediante flujos de Integración Continua (CI).
- **Calidad de Código:** Integrar el análisis estático de código mediante SonarCloud [24] para detectar vulnerabilidades, errores funcionales y mantener un bajo nivel de deuda técnica, siguiendo los estándares de la industria del *software*.

Conceptos teóricos

En aquellos proyectos que necesiten para su comprensión y desarrollo de unos conceptos teóricos de una determinada materia o de un determinado dominio de conocimiento, debe existir un apartado que sintetice dichos conceptos.

Algunos conceptos teóricos de $\text{\LaTeX}$ ¹.

3.1. Secciones

Las secciones se incluyen con el comando `section`.

Subsecciones

Además de secciones tenemos subsecciones.

Subsubsecciones

Y subsecciones.

3.2. Referencias

Las referencias se incluyen en el texto usando `cite` [29]. Para citar webs, artículos o libros [11].

¹Créditos a los proyectos de Álvaro López Cantero: Configurador de Presupuestos y Roberto Izquierdo Amo: PLQuiz

3.3. Imágenes

Se pueden incluir imágenes con los comandos standard de $\text{\LaTeX}$, pero esta plantilla dispone de comandos propios como por ejemplo el siguiente:



Figura 3.1: Autómata para una expresión vacía

3.4. Listas de items

Existen tres posibilidades:

- primer item.
- segundo item.

1. primer item.
2. segundo item.

Primer item más información sobre el primer item.

Segundo item más información sobre el segundo item.

■

Herramientas	App	AngularJS	API REST	BD	Memoria
HTML5		X			
CSS3		X			
BOOTSTRAP		X			
JavaScript		X			
AngularJS		X			
Bower		X			
PHP			X		
Karma + Jasmine		X			
Slim framework			X		
Idiorm			X		
Composer			X		
JSON		X	X		
PhpStorm		X	X		
MySQL				X	
PhpMyAdmin				X	
Git + BitBucket		X	X	X	X
MikTeX					X
TeXMaker					X
Astah					X
Balsamiq Mockups		X			
VersionOne		X	X	X	X

Tabla 3.1: Herramientas y tecnologías utilizadas en cada parte del proyecto

3.5. Tablas

Igualmente se pueden usar los comandos específicos de $\text{\LaTeX}$ o bien usar alguno de los comandos de la plantilla.

Técnicas y herramientas

4.1. Metodologías y técnicas de gestión

Para el desarrollo del presente proyecto se ha prescindido de los modelos tradicionales de ciclo de vida del *software* (como el modelo en Cascada), optando en su lugar por un enfoque ágil (*Agile*). Esta decisión metodológica se fundamenta en la necesidad de abordar requisitos en constante evolución y en la importancia de disponer de versiones funcionales del sistema desde las primeras etapas del desarrollo, algo crítico al integrar un simulador de telemetría y una aplicación de control en tiempo real.

Para materializar este enfoque, se ha aplicado una metodología pragmática que combina los principios iterativos de Scrum [22] con la gestión y trazabilidad de requisitos mediante *GitHub Issues*.

Scrum [22] y adaptación al entorno académico

Scrum [22] es un marco de trabajo ágil orientado a la gestión de proyectos complejos. Se basa en el desarrollo iterativo e incremental, donde el trabajo se estructura en ciclos de tiempo fijo denominados *sprints*. Durante cada *sprint*, el equipo se compromete a diseñar, implementar y probar un subconjunto de funcionalidades priorizadas, generando un incremento de *software* potencialmente entregable al final del ciclo.

Dada la naturaleza estrictamente individual de este Trabajo de Fin de Grado, el marco oficial de Scrum [22] requirió una adaptación metodológica para encajar en el contexto académico:

- ***Sprints* de dos semanas:** Se estableció una cadencia de desarrollo basada en iteraciones de catorce días. Esta ventana temporal propor-

cionó el equilibrio óptimo entre la obtención de resultados tangibles y la flexibilidad necesaria para absorber los picos de baja disponibilidad derivados del calendario académico y los periodos de exámenes.

- **Revisión de *Sprint* y validación de negocio:** Al finalizar cada iteración, se programaron reuniones de seguimiento con los tutores del proyecto. En este escenario, asumieron un rol análogo al de *Product Owner*, evaluando los incrementos funcionales de la aplicación, aportando retroalimentación técnica y validando que los requisitos cumplieran con los estándares de ingeniería exigidos.

Gestión de requisitos y trazabilidad con *GitHub Issues*

De forma complementaria a los ciclos de Scrum [22], se adoptó *GitHub Issues* como herramienta principal para el seguimiento operativo diario y la documentación de tareas.

Cada incidencia (*issue*) fue documentada detallando su propósito técnico y asignándole etiquetas (*labels*) estandarizadas que indicaban su prioridad y naturaleza funcional (por ejemplo, *testing* para añadir tests o *bug* para la resolución de fallos). Al estar estrechamente ligado al control de versiones, este enfoque facilitó asociar de forma automática las confirmaciones de código (*commits*).

Finalmente, la sincronización bidireccional entre *GitHub Issues* y la plataforma ágil Zube permitió agrupar estas incidencias para planificar los *sprints* de forma visual. Esta integración garantizó una trazabilidad absoluta del progreso, asegurando que el esfuerzo de programación estuviera siempre documentado y alineado con los objetivos de la iteración en curso.

4.2. Patrón de arquitectura

El diseño del sistema AgroSkopos se ha concebido bajo premisas de alta cohesión, bajo acoplamiento y reactividad en tiempo real, requisitos indispensables para una plataforma de telemetría robótica. Para satisfacer estas necesidades, el *software* se fundamenta en la combinación de varios patrones arquitectónicos complementarios.

Arquitectura Cliente-Servidor y *Single Page Application* (SPA)

A nivel macroarquitectónico, el sistema implementa un modelo Cliente-Servidor con un desacoplamiento estricto entre la interfaz de usuario (*frontend*) y la lógica de negocio (*backend*). A pesar de esta separación operativa e independiente, el código de ambas partes (`/app/client` y `/app/server`) se administra de manera centralizada mediante el patrón de Monorepo, lo que optimiza la gestión de dependencias y simplifica los flujos de integración continua sin romper el aislamiento técnico de cada capa.

El cliente web ha sido desarrollado siguiendo el patrón de Arquitectura de Una Sola Página (*Single Page Application* o SPA). A diferencia de las aplicaciones web tradicionales multi-página, la SPA carga un único documento HTML en el navegador y actualiza dinámicamente el contenido (el Modelo de Objetos del Documento o DOM) a medida que el usuario interactúa con la plataforma. Este enfoque reduce drásticamente el consumo de ancho de banda y proporciona una fluidez de navegación similar a la de una aplicación de escritorio, algo crítico al operar el mapa interactivo y el panel de control del robot.

Arquitectura en Capas (*Layered Architecture*) en el *Backend*

Para garantizar la mantenibilidad y escalabilidad del servidor, el *backend* implementa una Arquitectura en Capas lógicas. El flujo de información desde que entra una petición hasta que se consulta la base de datos atraviesa niveles con responsabilidades únicas y bien definidas:

1. **Capa de Enrutamiento (*Routes*):** Actúa como punto de entrada de la API REST, interceptando las peticiones HTTP (GET, POST, PUT, DELETE) y derivándolas al controlador correspondiente.
2. **Capa de Intermediación (*Middlewares*):** Subsistema transversal encargado de interceptar el tráfico para aplicar reglas de seguridad (verificación de *tokens* de autenticación) y gestión centralizada de errores antes de alcanzar la lógica principal.
3. **Capa de Aplicación (*Controllers*):** Contiene la lógica de negocio pura. Es la encargada de procesar las órdenes de control, gestionar el historial de misiones y preparar las respuestas para el cliente.

4. **Capa de Persistencia (*Data/Config*):** Abstrae la conexión y las consultas directas al sistema gestor de bases de datos relacional (PostgreSQL [25]).

Arquitectura Orientada a Eventos (*Event-Driven*)

Mientras que las operaciones de gestión (como consultar el historial de misiones o registrar usuarios) se resuelven mediante el clásico patrón de petición-respuesta (API REST), la monitorización cinemática y energética del robot exige un paradigma distinto.

Para solventar esta necesidad, AgroSkopos integra una Arquitectura Orientada a Eventos paralela a la API REST. Mediante el uso de WebSockets bidireccionales, el sistema establece un canal de comunicación persistente y de baja latencia. El simulador del robot actúa como un "Productor" de eventos (emitiendo variables de estado como batería, velocidad y coordenadas cada pocos milisegundos), el servidor actúa como un "*Broker*." despachador, y los componentes visuales del *frontend* actúan como "Consumidores", reaccionando y re-renderizando la pantalla instantáneamente tras la recepción de un nuevo evento. Este patrón es el núcleo que permite que el visor funcione en estricto tiempo real.

4.3. Tecnologías y *frameworks* principales

En esta sección se detallan las herramientas tecnológicas que conforman el ecosistema del proyecto, divididas según la capa arquitectónica en la que operan.

Backend (Lógica de servidor)

- **Node.js [19] y Express [18]:** Node.js [19] es un entorno de ejecución multiplataforma basado en el motor V8 de JavaScript, diseñado para construir aplicaciones de red escalables mediante un modelo de entrada/salida sin bloqueo y orientado a eventos. Sobre este entorno se emplea Express [18], un *framework* web minimalista que facilita la creación de la API REST, gestionando el enrutamiento de las peticiones HTTP (rutas de usuarios, misiones y control) y la integración de *middlewares* de forma estructurada.
- **PostgreSQL [25] y node-postgres [5] (pg):** PostgreSQL [25] es un sistema de gestión de bases de datos relacional orientado a objetos,

conocido por su robustez, extensibilidad y cumplimiento del estándar SQL. Constituye el núcleo de persistencia del sistema, almacenando el historial de misiones, telemetría y perfiles de usuario. Para la comunicación entre la aplicación Node.js [19] y la base de datos se utiliza pg (*node-postgres* [5]), un cliente de PostgreSQL [25] que permite la ejecución asíncrona de consultas parametrizadas, protegiendo al sistema contra inyecciones SQL.

- **Socket.io [23] (Servidor):** Socket.io [23] es una biblioteca que habilita la comunicación bidireccional, persistente y de baja latencia entre el cliente web y el servidor. En el contexto de AgroSkopos, actúa como el motor de la arquitectura orientada a eventos, permitiendo la transmisión en tiempo real de la telemetría del robot (coordenadas, velocidad, batería) sin la sobrecarga de cabeceras de las peticiones HTTP tradicionales.
- **Seguridad de la API (JWT, Bcrypt, Helmet y Rate Limit):** Para garantizar la integridad y confidencialidad de la API, se ha implementado un ecosistema de seguridad compuesto por varias librerías:
 - **JSON Web Token [3] (jsonwebtoken):** Estándar abierto (RFC 7519) utilizado para transmitir información de forma segura entre el cliente y el servidor en formato JSON. Se emplea para la autenticación sin estado (*stateless*) tras el inicio de sesión.
 - **Bcrypt:** Librería de cifrado basada en el algoritmo de *hash* de contraseñas de Blowfish. Garantiza que las credenciales de los usuarios se almacenen de forma segura y unidireccional.
 - **Helmet y Express [18]-Rate-Limit:** *Middlewares* de seguridad que protegen la aplicación configurando adecuadamente las cabeceras HTTP y limitando el número máximo de peticiones por IP, mitigando ataques de denegación de servicio (DDoS) y ataques de fuerza bruta.
- **Turf.js [26]:** Turf.js [26] es una biblioteca avanzada de análisis espacial en JavaScript. En el servidor, se emplea para realizar operaciones matemáticas geoespaciales complejas de forma nativa, tales como la validación de que las coordenadas del robot se encuentran dentro de los polígonos permitidos (geocercas) o el cálculo de distancias entre *waypoints* durante la simulación cinemática.

Frontend (Interfaz de usuario)

- **React [15] y Vite [30]:** React [15] es una biblioteca de JavaScript desarrollada por Meta para la construcción de interfaces de usuario basadas en componentes declarativos y reutilizables. Su uso permite gestionar dinámicamente el DOM mediante un modelo de renderizado reactivo. Para la construcción y empaquetado del código se emplea Vite [30], una herramienta de desarrollo de nueva generación que aprovecha los módulos ES nativos del navegador para ofrecer un entorno de desarrollo ultrarrápido y compilaciones optimizadas.
- **React [15] Router:** React [15] Router es la biblioteca estándar de enrutamiento para aplicaciones React [15]. Habilita la navegación dinámica entre distintas vistas del panel de control (*Dashboard*, Mapa, Historial, Perfil) sin necesidad de recargar la página web, consolidando la arquitectura de *Single Page Application* (SPA).
- **Zustand [20]:** Zustand [20] es un gestor de estado global ligero, rápido y escalable. A diferencia de soluciones más pesadas como Redux, Zustand [20] utiliza *hooks* de React [15] para compartir el estado (como los datos telemétricos en vivo o el usuario activo) entre componentes profundamente anidados sin incurrir en problemas de renderizado innecesario ni excesivo código repetitivo (*prop-drilling*).
- **Ecosistema Leaflet [1] (Geoman y Heatmaps):** Leaflet [1] es la principal biblioteca de código abierto para la creación de mapas interactivos adaptables a dispositivos móviles. Para su integración reactiva se emplea React [15]-Leaflet [1]. Este núcleo cartográfico se ha potenciado con dos extensiones fundamentales para el sector agrícola:
 - **Leaflet [1]-Geoman:** Herramienta que proporciona controles avanzados de dibujo y edición geométrica, permitiendo al usuario trazar polígonos (parcelas) y rutas directamente sobre el visor espacial.
 - **Leaflet [1].heat:** *Plugin* para la generación de mapas de calor, utilizado para representar gráficamente la densidad de datos edafológicos (como humedad o nutrientes) sobre el terreno.
- **Axios [4]:** Axios [4] es un cliente HTTP basado en promesas. Se encarga de gestionar la comunicación asíncrona entre la interfaz de React [15] y la API REST del *backend*, facilitando la interceptación de peticiones para la inyección automática de *tokens* de seguridad y la gestión centralizada de errores de red.

- **Recharts [21]:** Recharts [21] es una biblioteca de visualización de datos basada en componentes de React [15] y D3.js. Se utiliza para modelar la información numérica del sistema mediante gráficos interactivos y adaptables (como la curva de descarga energética de la batería o las estadísticas de rendimiento de las misiones).
- **Internacionalización (i18next [9]):** Para dotar al sistema de soporte multilingüe, se ha integrado el *framework* i18next [9] junto a su conector react-i18next [9]. A su vez, se emplea un detector de idioma en el navegador que adapta automáticamente la interfaz del visor a la configuración regional del operador.
- **PWA (*Progressive Web App*):** Mediante el *plugin* vite-plugin-pwa [27], la interfaz está configurada como una Aplicación Web Progresiva (PWA). Esto permite la instalación nativa de AgroSkopos en dispositivos móviles, tabletas y ordenadores. El *plugin* genera y registra automáticamente *Service Workers* que cachean los recursos estáticos del cliente (*App Shell*). Esto mejora drásticamente los tiempos de carga en zonas rurales con baja cobertura, reduce el consumo de datos y sienta la base tecnológica necesaria para implementar capacidades operativas *offline* más avanzadas en futuras iteraciones del proyecto.

Testing y Calidad del *Software*

Para asegurar la robustez de la plataforma, se emplean dos ecosistemas de pruebas consolidados:

- **Backend:** Se utiliza Jest [14] como marco de pruebas unitarias para aislar la lógica de controladores, y Supertest [12] para la validación *End-to-End* (E2E) de las rutas HTTP. Además, se integra Testcontainers [2] para levantar instancias efímeras y aisladas de PostgreSQL [25] en Docker [7] durante la ejecución de las pruebas de integración.
- **Frontend:** Se emplea Vitest [28] (un marco de pruebas nativo de Vite [30]) en conjunción con React [15] Testing Library y jsdom [10] para simular el comportamiento del navegador y validar la correcta renderización e interactividad de los componentes visuales.

Contenedores y orquestación

Docker [7] y Docker [7] Compose [7] conforman una plataforma de contenerización que permite empaquetar aplicaciones junto con sus dependencias

y variables de entorno en unidades estándar aisladas. Para la orquestación del ecosistema completo (servidor Node.js [19], cliente React [15] servido mediante Nginx, y base de datos PostgreSQL [25]) se emplea Docker [7] Compose. Esta herramienta permite levantar toda la arquitectura con un único archivo declarativo, garantizando que el entorno de desarrollo sea idéntico al entorno de producción.

Automatizaciones y Análisis estático

- **ESLint [17] y SonarCloud [24]:** ESLint [17] es una herramienta de análisis estático (*linter*) que identifica y reporta patrones problemáticos en el código JavaScript, garantizando el cumplimiento de los estándares de estilo. Posteriormente, este código es sometido a un análisis de calidad profundo y seguridad continua mediante la integración de SonarCloud [24].
- **GitHub Actions [8]:** Herramienta de Integración Continua (CI) [8] que automatiza los flujos de trabajo del repositorio. En este proyecto, orquesta la ejecución automática de los *tests* y el envío de reportes a SonarCloud [24] cada vez que se detectan cambios en el código, bloqueando la integración de código defectuoso.

Documentación y diseño

Para la redacción, estructuración y composición tipográfica de la presente memoria técnica se ha utilizado inicialmente el sistema L^AT_EX [13] a través de la plataforma web Overleaf [6]. La elección de estas herramientas responde a su excelente rendimiento en el manejo de documentos de carácter científico-técnico y a su capacidad para garantizar un formato limpio, uniforme y profesional. En una primera fase, Overleaf facilitó la automatización de elementos complejos como los índices dinámicos, las referencias cruzadas, el listado de figuras y la gestión bibliográfica.

A pesar de tratarse de un desarrollo de *software* individual, la plataforma web resultó fundamental para optimizar el flujo de trabajo inicial con los tutores del proyecto, habilitando un acceso compartido para la supervisión asíncrona de los borradores en tiempo real.

No obstante, en las etapas más avanzadas de la redacción, y por una estricta cuestión de comodidad y centralización del entorno de trabajo, se procedió a descargar e instalar una distribución de L^AT_EX de forma local en el equipo de desarrollo. A partir de este momento, el grueso de la documentación

continuó redactándose y compilándose directamente desde **Visual Studio Code** [16], aprovechando la potente extensión **LaTeX Workshop** [31]. Este cambio permitió integrar el flujo de escritura de la memoria con el mismo editor utilizado para el desarrollo del código fuente, ganando agilidad y unificando todo el proyecto en un único entorno local.

4.4. Gestión de código y entorno de desarrollo

Control de versiones y flujo de trabajo individual

El control de versiones del proyecto se ha gestionado mediante Git, empleando GitHub como repositorio remoto centralizado y plataforma de respaldo para el código fuente. Al tratarse de un desarrollo de *software* individual y no colaborativo, se descartó el uso de modelos de ramificación complejos (como GitFlow), los cuales añaden una sobrecarga innecesaria de fusión de ramas (*merging*) y resolución de conflictos.

En su lugar, se adoptó un flujo de trabajo simplificado enfocado a la trazabilidad y la estabilidad del código. El desarrollo se centralizó directamente sobre la rama principal o mediante ramas efímeras de corta duración vinculadas a tareas concretas. Este enfoque lineal garantizó que el historial de desarrollo permaneciera limpio y comprensible, asegurando que cada incremento de funcionalidad fuera registrado cronológicamente sin comprometer la velocidad de despliegue ni la integridad de la aplicación.

Conventional Commits Para mantener la disciplina y el rigor metodológico en el repositorio, la confirmación de cambios se ha regido por la especificación semántica de *Conventional Commits*. Cada mensaje de registro (*commit*) se ha estructurado utilizando prefijos obligatorios que identifican unívocamente la naturaleza de la modificación, tales como "feat" para nuevas características, "fix" para correcciones de errores o "test" para la infraestructura de pruebas. Esta práctica ha dotado al proyecto de una legibilidad excelente, facilitando la auditoría del *software* y la generación de un historial de cambios profesional.

Entorno operativo y administración

El Entorno de Desarrollo Integrado (IDE) principal escogido para la codificación de toda la plataforma ha sido Visual Studio Code, debido a su ligereza, naturaleza de código abierto y su soporte nativo para el ecosistema

de JavaScript y Node.js [19]. Para optimizar el ciclo de desarrollo y asegurar la homogeneidad del *software*, la potencia del editor se ha ampliado mediante un conjunto seleccionado de extensiones técnicas:

- **ESLint [17] y Prettier:** Encargados de realizar el análisis estático y el formateo en tiempo de ejecución dentro del editor. Forzaron el cumplimiento de las reglas de estilo y detectaron errores de sintaxis o variables huérfanas antes de la compilación.
- **Docker [7] y Containers (Microsoft):** Extensiones para la inspección visual, gestión y monitorización en tiempo real de los contenedores locales (*Frontend*, *Backend* y Base de Datos), agilizando las pruebas de despliegue y la orquestación del entorno.
- **SonarLint:** Integrado como un *linter* avanzado en el IDE, conectándose de forma bidireccional con las reglas de calidad de SonarCloud [24] para detectar vulnerabilidades de seguridad (*Security Hotspots*) y deuda técnica en el mismo instante de la escritura del código.
- **Database Client for PostgreSQL [25]:** Herramienta integrada en el propio editor que permitió la conexión directa a la base de datos local y a los contenedores, facilitando la ejecución de sentencias SQL y la validación del esquema de datos sin necesidad de recurrir a clientes pesados externos.

Aspectos relevantes del desarrollo del proyecto

En este capítulo se describe el ciclo de vida completo del proyecto AgroS-kopos, abordando desde su concepción inicial y planificación metodológica, hasta las fases de implementación técnica y validación.

5.1. Motivación del proyecto

El origen de este Trabajo de Fin de Grado surge del interés por aplicar y consolidar los conocimientos adquiridos en la titulación en un entorno práctico multidisciplinar. La motivación principal fue el diseño de una plataforma de telemetría y control agrícola capaz de integrarse de forma colaborativa con otro Trabajo de Fin de Grado perteneciente a la rama de Ingeniería Electrónica. Mientras que dicho proyecto abordaba el diseño, ensamblaje y la programación a bajo nivel del chasis físico de un robot agrícola, el presente trabajo asumió el reto de desarrollar de forma íntegra toda la capa de *software* de alto nivel.

Esto incluía la creación de la infraestructura de comunicaciones, la persistencia de datos y la interfaz de usuario que permitiría gobernar dicho vehículo de forma remota. Esta propuesta representó un desafío técnico sumamente enriquecedor, al permitir la convergencia metodológica entre el desarrollo de aplicaciones web modernas orientadas a la baja latencia y la interacción directa con el *hardware* en el ámbito del Internet de las Cosas (IoT).

5.2. Inicio del proyecto y evolución del alcance

En la fase inicial de toma de requisitos, planteada a principio de septiembre, el alcance del proyecto era considerablemente más reducido y lineal. El objetivo original consistía en desarrollar una aplicación funcional que cubriera la gestión básica de usuarios y permitiría visualizar la telemetría del robot en tiempo real. En cuanto al control, únicamente se contemplaba una navegación rudimentaria mediante el envío de coordenadas individuales basadas en los clics del operador sobre un mapa, complementado con tablas y gráficas estáticas para el análisis de los datos.

Sin embargo, durante el transcurso del curso académico, el proyecto experimentó una reestructuración forzosa. Diversos problemas técnicos, logísticos y de suministro impidieron que el proyecto de Ingeniería Electrónica pudiera finalizar la construcción del robot físico a tiempo. Esta eventualidad imposibilitó la fase de integración de *hardware* que estaba prevista inicialmente.

Lejos de suponer un bloqueo, este contratiempo impulsó una ambiciosa evolución orgánica del *software*. Para poder validar la arquitectura orientada a eventos y la recepción de telemetría, el motor de simulación interno (que inicialmente se había concebido como un simple *script* de pruebas temporales) se reescribió por completo y se dotó de algoritmos cinemáticos avanzados. Al no requerir la inversión de tiempo en la integración física, el esfuerzo de desarrollo se redirigió hacia la ampliación masiva de las funcionalidades de la plataforma, convirtiendo lo que iba a ser un simple visor de datos en un producto tecnológico integral mucho más completo.

De este modo, se incorporó un complejo sistema de misiones autónomas y planificación de rutas, delimitación virtual del terreno (geocercas), soporte multilenguaje (i18n), personalización de la interfaz (modo oscuro) y la migración arquitectónica hacia una Aplicación Web Progresiva (PWA).

5.3. Planificación

La planificación del proyecto se estructuró asumiendo la alta incertidumbre derivada de un desarrollo dependiente, en sus inicios, de factores externos (el *hardware*). Como se documentó en el capítulo metodológico, se empleó Scrum [22] junto a la herramienta Zube para orquestrar el traba-

jo en iteraciones de dos semanas, lo que permitió una gran capacidad de adaptación ante los cambios de alcance.

El ciclo de desarrollo se dividió en las siguientes grandes fases planificadas:

1. **Fase de Análisis y Diseño:** Toma de requisitos iniciales, diseño de la base de datos relacional y elaboración de los prototipos de la interfaz gráfica de usuario.
2. **Fase de Desarrollo *Frontend*:** Construcción de la *Single Page Application* (SPA) con React [15], integración del mapa interactivo con Leaflet [1] y conexión con los servicios del *backend*.
3. **Fase de Desarrollo *Backend*:** Implementación del servidor Node.js [19], despliegue del contenedor PostgreSQL [25], configuración de la API REST y establecimiento del servidor de WebSockets para la comunicación bidireccional.
4. **Fase de Refactorización y Simulación Avanzada:** Hito introducido tras la cancelación de las pruebas físicas. Consistió en el desarrollo del motor de simulación cinemática y la generación dinámica de datos agronómicos (`simulator.js`) para replicar el comportamiento de un entorno real.
5. **Fase de Expansión de Funcionalidades:** Implementación de las misiones, algoritmos de navegación (rutas de cobertura y *zigzag*), internacionalización y configuración de la PWA.
6. **Fase de Pruebas (*Testing*) y Despliegue:** Consolidación de las pruebas unitarias y E2E, análisis estático con SonarCloud [24] y contenedorización final mediante Docker [7] Compose [7].

Esta fragmentación en fases permitió aislar la complejidad, garantizando que el núcleo de la aplicación estuviera consolidado y probado antes de implementar las capas superiores de lógica de negocio y visualización.

5.4. Adquisición de conceptos técnicos

El desarrollo de una aplicación con requerimientos de telemetría, simulación cinemática y mapas interactivos en tiempo real supuso un desafío técnico significativo, exigiendo un periodo intensivo de capacitación tecnológica previo a la fase de codificación. Puesto que el *stack* tecnológico

empleado (PERN: PostgreSQL [25], Express [18], React [15] y Node.js [19]) difería notablemente de las tecnologías abordadas con profundidad a lo largo del plan de estudios del grado, el proyecto requirió un firme proceso de investigación y autoaprendizaje.

Esta adquisición de competencias se llevó a cabo en dos etapas diferenciadas:

- **Etapla Inicial (Verano 2025):** El proceso de formación comenzó durante el periodo estival, coincidiendo con las prácticas extracurriculares en empresa. En esta fase se llevó a cabo un estudio autodidacta centrado en los fundamentos del desarrollo web moderno, asimilando la sintaxis y capacidades de JavaScript (ES6+), así como la estructuración y diseño con HTML5 y CSS3. Esta etapa sentó las bases necesarias para abordar la programación orientada a la web con la agilidad requerida.
- **Etapla de Pre-desarrollo (Septiembre - Octubre 2025):** Tras la formalización de los requisitos del proyecto, se estableció un bloque de un mes dedicado de forma exclusiva a la investigación teórica, antes de realizar la primera confirmación de código (*commit*). Durante este tiempo se consolidó el dominio del ecosistema de React [15] y de las tecnologías fundamentales para AgroSkopos:
 - **Paradigma de React [15] y Estado Global:** Se profundizó en el modelo declarativo, el ciclo de vida de los componentes y la gestión avanzada de estados globales mediante *hooks* optimizados como Zustand [20], previendo el alto volumen de datos de telemetría que la aplicación debía manejar.
 - **Librerías Geoespaciales (Leaflet [1]):** Sabiendo que el núcleo visual del proyecto iba a ser un visor interactivo de precisión, se realizó un estudio pormenorizado de la API de Leaflet [1]. Se analizaron conceptos cartográficos como las capas de teselas (*tile layers*), las proyecciones de coordenadas y la renderización reactiva de polígonos y marcadores.

Esta fase preparatoria resultó crítica para mitigar los riesgos técnicos del proyecto. El tiempo invertido garantizó que, una vez iniciado el desarrollo *frontend*, se ejecutara bajo decisiones de diseño de *software* maduras, limpias y escalables, evitando la deuda técnica temprana.

5.5. Desarrollo del proyecto

La implementación técnica del sistema AgroSkopos se llevó a cabo de manera incremental, siguiendo la metodología ágil descrita. El esfuerzo se estructuró a lo largo de seis fases lógicas, divididas operativamente entre el *frontend* y el *backend*:

Evolución del *Frontend* y la Interfaz

Fase 1: Cimentación Arquitectónica y Prototipado Visual (Octubre 2025) El desarrollo dio comienzo con la configuración del entorno *frontend* (React [15] y Vite [30]) y el diseño de las interfaces base. Durante los primeros *sprints*, el trabajo se centró en construir la estructura esquelética de la *Single Page Application* (SPA), implementando el enrutamiento (React [15] Router), la pantalla de autenticación y el panel principal (*Dashboard*). En esta etapa temprana, se integró por primera vez el visor cartográfico mediante Leaflet [1] y se estableció el sistema de gestión de estado global utilizando Zustand [20]. Las primeras iteraciones finalizaron operando en este punto con datos estáticos simulados (*Mocks*).

Evolución del *Backend* y Lógica de Servidor

Fase 2: Arquitectura *Backend*, Seguridad y Modelado de Datos (Noviembre 2025) Una vez estabilizada la interfaz, el enfoque se trasladó al servidor Node.js [19]. Se abandonaron los datos estáticos migrando el núcleo de almacenamiento a PostgreSQL [25]. En esta fase se implementaron los pilares de seguridad del sistema: cifrado de contraseñas mediante Bcrypt, protección de rutas de la API, y un robusto sistema de autenticación basado en *JSON Web Tokens* (JWT). Asimismo, se desarrolló la lógica de control de acceso basado en roles (RBAC).

Integración de Sistemas y Simulación

Fase 3: El Punto de Inflexión - Simulación y Telemetría en Tiempo Real (Finales de Noviembre 2025) Ante la imposibilidad de integrar el chasis del robot físico, el desarrollo pivotó estratégicamente hacia la creación de un simulador telemétrico propio en el *backend*. Se sustituyeron las peticiones HTTP convencionales por una arquitectura orientada a eventos mediante WebSockets, logrando comunicación bidireccional en tiempo real y habilitando notificaciones visuales en el *frontend*.

Fase 4: Control Cinemático y Analítica Espacial (Diciembre 2025 - Febrero 2026) Se desarrollaron las interfaces y la lógica del servidor para permitir el control manual del vehículo y se programaron los primeros algoritmos de navegación automatizada (rutas de cobertura y *zigzag*). A nivel espacial, se introdujo el concepto de *Geofencing* (delimitación poligonal de zonas seguras) y se implementaron mapas de calor (*Heatmaps*) para visualizar la densidad de nutrientes sobre el terreno.

Fase 5: Misiones Autónomas e Internacionalización (Marzo - Abril 2026) Se diseñó el esquema de la base de datos y la API para dar soporte a un sistema completo de Misiones. Se implementó una interfaz de creación, gestión y ejecución de misiones en vivo, apoyada por una máquina de estados en el *backend*. Además, se integró el *framework* *i18next* [9] para soportar la internacionalización (multi-idioma).

Fase 6: Despliegue, PWA y Aseguramiento de la Calidad (*Testing*) (Abril - Mayo 2026) El repositorio se reestructuró bajo un modelo Monorepo estricto y se configuraron los contenedores Docker [7] (*docker-compose*). A nivel de cliente, la aplicación se transformó en una Aplicación Web Progresiva (PWA). Finalmente, se diseñó e implementó la cobertura de pruebas (*Testing*), abarcando pruebas unitarias de controladores e integración *End-to-End* (E2E).

5.6. Testing y Calidad de Código

Para garantizar la estabilidad, robustez y correcta funcionalidad del sistema AgroSkopos, se diseñó e implementó una estrategia integral de pruebas automáticas que abarca tanto el cliente web como el servidor.

1. **Pruebas Unitarias y de Componentes en el *Frontend*:** Para la interfaz de usuario se empleó Vitest [28] en combinación con *React* [15] *Testing Library*. Se aislaron los componentes complejos mediante el uso intensivo de *Mocks* (simulaciones) para dependencias externas pesadas. Se validó el comportamiento asíncrono y los cambios en el estado global (Zustand [20]), así como los contextos de la aplicación (*Context API*).
2. **Pruebas Unitarias lógicas en el *Backend* (Jest [14]):** En la capa del servidor, las pruebas unitarias se enfocaron en aislar los controladores lógicos (*Controllers*) de la infraestructura externa. Empleando

Jest [14], se desarrollaron baterías de pruebas que simulaban (*mockeban*) tanto las consultas a la base de datos PostgreSQL [25] como las operaciones criptográficas.

3. **Pruebas *End-to-End* (E2E) y Testcontainers [2] en la API REST:** El nivel más avanzado de la estrategia de calidad del *backend* fue la validación completa del ciclo de vida de las peticiones HTTP mediante entornos efímeros. Para ello, se integró Testcontainers [2], orquestando de forma aislada una instancia de PostgreSQL [25] en Docker [7] para cada batería de pruebas automatizadas mediante Supertest [12], garantizando el aislamiento (idempotencia) entre cada prueba.

5.7. Despliegue

Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

Conclusiones y líneas de trabajo futuras

7.1. Conclusiones

El desarrollo de este Trabajo de Fin de Grado ha terminado de manera satisfactoria, logrando el cumplimiento de los objetivos propuestos desde su planificación. Con AgroSkopos, se ha materializado una plataforma web centralizada que da respuesta a las necesidades de monitorización, telemetría y control remoto en el ámbito de la agricultura de precisión. Mediante una arquitectura de baja latencia, se han satisfecho con éxito los requisitos funcionales exigidos: desde la implementación de un mapa interactivo y la gestión de misiones autónomas, hasta el despliegue de un panel de telemetría y control manual en tiempo real. Además, la adopción de metodologías ágiles y herramientas de análisis estático ha garantizado un producto final robusto, contenerizado y con altos estándares de calidad.

Desde la perspectiva de la ingeniería del *software*, el proyecto ha representado un valioso ejercicio de diseño arquitectónico y resolución de problemas complejos. El principal reto superado ha sido la construcción de una infraestructura escalable y de baja latencia, orquestando con éxito la comunicación bidireccional en tiempo real (WebSockets), la gestión eficiente del estado en el cliente y la automatización de pruebas mediante flujos de integración continua (CI/CD). Esta robustez estructural fue precisamente la que permitió una rápida adaptación técnica ante el retraso logístico del *hardware*.

En el plano formativo, este proyecto ha supuesto un cierre perfecto para mi formación en el grado. Asumir en solitario la responsabilidad integral del ciclo de vida del *software*, desde la toma de requisitos hasta las pruebas

automatizadas, ha permitido consolidar y poner en práctica de manera transversal los conocimientos teóricos adquiridos a lo largo de la carrera. Simultáneamente, el proyecto ha motivado un enriquecedor proceso de autoaprendizaje, fundamental para dominar y aplicar tecnologías modernas ampliamente demandadas en el sector.

En definitiva, AgroSkopos se consolida como una herramienta robusta, funcional y escalable orientada a la digitalización del sector agrario.

7.2. Líneas de trabajo futuras

El desarrollo y la posterior evaluación de AgroSkopos han puesto de manifiesto la viabilidad de la plataforma como núcleo tecnológico operativo, abriendo a su vez nuevas e interesantes vías de investigación e ingeniería para enriquecer el sistema en futuras iteraciones.

Transición a *Hardware* Real y Comunicaciones Avanzadas

El sistema actual se apoya de forma robusta en un simulador nativo diseñado para replicar la dinámica del entorno real. El siguiente paso natural en la evolución del proyecto es sustituir este motor de *software* por una pasarela IoT (*Internet of Things*) que establezca comunicación directa con el chasis de un robot físico real. Para lograrlo, se propone el uso del estándar *Robot Operating System* (ROS2) integrado con MQTT, lo que garantizaría un flujo telemétrico eficiente.

Además, actualmente el visor de cámara emplea una simulación funcional integrada en la interfaz; la mejora propuesta consiste en implementar un servidor de *streaming* nativo basado en la tecnología WebRTC. Esto permitirá transmitir el flujo de vídeo en vivo directamente desde las cámaras del robot hacia el panel de control del operador con latencias sub-segundo, un factor crítico e indispensable para asegurar la maniobrabilidad durante la operación manual a distancia.

Por otra parte, se plantea adaptar la arquitectura del sistema para soportar operaciones sin conexión en entornos agrícolas caracterizados por conectividad intermitente. La solución pasaría por dotar al cliente de la capacidad de almacenar localmente tanto la telemetría generada como el registro temporal de misiones y captura de datos. Al recuperar la señal de red, el sistema orquestrará una sincronización diferida automática contra la base

de datos central en PostgreSQL [25], asegurando la integridad transaccional de los datos agrícolas sin riesgo de pérdida.

Visión Artificial, Realidad Aumentada y Registro Visual

Aprovechando el flujo continuo de vídeo en tiempo real implementado en el control remoto, el ecosistema puede beneficiarse enormemente de la integración de modelos de *Machine Learning* (tales como la arquitectura YOLOv8). Al procesar las imágenes en el propio extremo de la red (*Edge Computing*), la inteligencia artificial sería capaz de identificar amenazas agronómicas, como la presencia de malas hierbas o indicadores visuales de enfermedades en las plantaciones.

Mediante el uso de librerías nativas del navegador web como WebGL o Canvas API, la información detectada por la IA se proyectaría directamente sobre el vídeo simulando una interfaz de tipo HUD (*Head-Up Display*). Este enfoque enriquecería sustancialmente la experiencia del operador, permitiéndole no solo observar el terreno a través de la cámara, sino también visualizar marcadores geográficos precisos, trazados dinámicos de rutas, delimitaciones de geocercas y alertas tempranas, multiplicando la conciencia situacional en tiempo real.

Complementariamente, el sistema integraría un módulo de captura y almacenamiento geolocalizado de imágenes. Esto facilitaría el trabajo de ingeniería agrícola al permitir la configuración autónoma del robot para registrar el estado visual del cultivo fotograma a fotograma en los distintos puntos de recolección de datos (*waypoints*), generando en la base de datos un inventario y un rastro visual persistente asociado a las coordenadas GPS exactas.

Planificación Óptima y Control Cinemático Avanzado

Aunque los algoritmos de navegación actuales garantizan una cobertura de área funcional (mediante patrones estáticos en *zigzag*), existe un gran margen de mejora hacia la planificación adaptativa inteligente. El sistema evolucionaría con la introducción de algoritmos de búsqueda heurística de grafos, tales como A* o D* Lite, los cuales se aplicarían no sobre un plano liso, sino sobre un sofisticado "mapa de costes" topográfico. La IA evaluaría en tiempo real diversas condiciones críticas del entorno —desde la orografía y el ángulo de inclinación del terreno, hasta la densidad del suelo— para trazar

la ruta óptima de menor esfuerzo cinemático, reduciendo drásticamente tanto el tiempo empleado por misión como el consumo energético total.

En escenarios reales de operación, la seguridad del robot frente a imprevistos es ineludible. Por este motivo, la plataforma debe integrar en sus algoritmos el procesamiento en bruto procedente de escáneres LIDAR y sensores de proximidad ultrasonidos. La aplicación de técnicas de evasión dinámica de obstáculos (como los Histogramas de Campo Vectorial) facultaría al robot agrícola para sortear automáticamente maquinaria u operarios que interfirieran accidentalmente en su trayectoria predefinida, garantizando una maniobra segura y la posterior corrección automatizada hacia el objetivo original.

Escalabilidad hacia la Gestión de Flotas (*Swarm Robotics*)

AgroSkopos, en su versión fundacional, ha optimizado su capa de servicios y comunicación de eventos asumiendo un escenario operativo de un único vehículo agrícola. No obstante, la robustez de su motor de bases de datos PostgreSQL [25] y su canal de WebSockets soportan teóricamente arquitecturas de mayor densidad y demanda de flujo. La proyección hacia el futuro de esta herramienta radicaría en rediseñar la plataforma de control como un administrador de flotas centralizado, dotando a la interfaz de las capacidades de monitorización múltiple y en paralelo. Esta evolución orquestaría escenarios de cooperación avanzada, posibilitando a un único operador humano la supervisión general y la delegación de tareas concurrentes a múltiples robots agrícolas (*enjambres*) operando en sincronía a lo largo de extensiones de cultivo a gran escala.

Experiencia de Usuario y Seguridad Avanzada

Por último, para dotar al producto de un estándar organizativo acorde a sistemas industriales corporativos, el proceso de autenticación de usuarios podría enriquecerse. Incorporar capacidades de seguridad suplementarias vinculadas a las cuentas de perfil, tales como protocolos de Autenticación de Doble Factor (2FA) o manejo automatizado de alertas de incidencias directamente remitidas a canales asíncronos. La implementación de un módulo de notificaciones externas posibilitaría enviar de forma transparente, a través de correos electrónicos, SMS o integración con plataformas de mensajería externa, alertas críticas que superen el marco cerrado de la aplicación web. De esta manera, el usuario supervisor recibiría en todo

momento advertencias sobre escenarios de alto riesgo, como descensos de batería bajo umbrales seguros o caídas prolongadas del enlace telemétrico con el vehículo de campo.

Bibliografía

- [1] Vladimir Agafonkin. Leaflet: an open-source javascript library for mobile-friendly interactive maps. <https://leafletjs.com/>, 2026.
- [2] AtomicJar, Inc. Testcontainers. <https://testcontainers.com/>, 2026.
- [3] Auth0. Json web tokens. <https://jwt.io/>, 2026.
- [4] Axios. Axios: Promise based http client for the browser and node.js. <https://axios-http.com/>, 2026.
- [5] B. Carlson. node-postgres. <https://node-postgres.com/>, 2026.
- [6] Digital Science. Overleaf: Collaborative latex editor. <https://www.overleaf.com/>, 2026.
- [7] Docker Inc. Docker: Accelerated container application development. <https://www.docker.com/>, 2026.
- [8] GitHub, Inc. Github actions. <https://github.com/features/actions>, 2026.
- [9] i18next. i18next: internationalization-framework. <https://www.i18next.com/>, 2026.
- [10] jsdom. jsdom: A javascript implementation of various web standards. <https://github.com/jsdom/jsdom>, 2026.
- [11] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.
- [12] T. Lad. Supertest. <https://github.com/ladjs/supertest>, 2026.

- [13] Leslie Lamport. Latex: A document preparation system. <https://www.latex-project.org/>, 1985.
- [14] Meta Platforms, Inc. Jest: Delightful javascript testing. <https://jestjs.io/>, 2026.
- [15] Meta Platforms, Inc. React: The library for web and native user interfaces. <https://react.dev/>, 2026.
- [16] Microsoft Corporation. Visual studio code. <https://code.visualstudio.com/>, 2026.
- [17] OpenJS Foundation. Eslint - pluggable javascript linter. <https://eslint.org/>, 2026.
- [18] OpenJS Foundation. Express - node.js web application framework. <https://expressjs.com/>, 2026.
- [19] OpenJS Foundation. Node.js. <https://nodejs.org/>, 2026.
- [20] Poimandres. Zustand: Bear necessities for state management in react. <https://zustand-demo.pmnd.rs/>, 2026.
- [21] Recharts. Recharts: A composable charting library built on react components. <https://recharts.org/>, 2026.
- [22] Ken Schwaber and Jeff Sutherland. The scrum guide. <https://scrumguides.org/>, 2020.
- [23] Socket.IO. Socket.io. <https://socket.io/>, 2026.
- [24] SonarSource SA. Sonarqube / sonarcloud. <https://www.sonarsource.com/>, 2026.
- [25] The PostgreSQL Global Development Group. Postgresql: The world's most advanced open source relational database. <https://www.postgresql.org/>, 2026.
- [26] Turf.js. Turf.js - advanced geospatial analysis for browsers and node.js. <https://turfjs.org/>, 2026.
- [27] Vite PWA. Vite plugin pwa. <https://vite-pwa-org.netlify.app/>, 2026.
- [28] Vitest Contributors. Vitest: A vite-native testing framework. <https://vitest.dev/>, 2026.

- [29] Wikipedia. Latex — wikipedia, la enciclopedia libre. <https://es.wikipedia.org/w/index.php?title=LaTeX&oldid=84209252>, 2015.
- [30] Evan You. Vite: Next generation frontend tooling. <https://vitejs.dev/>, 2026.
- [31] James Yu. Latex workshop extension for visual studio code. <https://marketplace.visualstudio.com/items?itemName=James-Yu.latex-workshop>, 2026.